

宁波大学
NINGBO UNIVERSITY

实 验 报 告

[实验 5 串口通信实验]

姓名：	赵瑞杰
学号：	216001944
课程名称：	嵌入式系统
所在学院：	信息学院
专业班级：	计算机科学与技术 1 班
任课教师：	王翀 王艺睿

实验项目名称	串口通信实验

一、实验目的与要求：

将利用 EXYNOS4412 的复用引脚 XuRXD2、XuTXD2 这 2 个引脚收发串口上的数据，实现串口调试助手上显示数据。

二、实验环境：

Windows 7 操作系统、ARM Cortex-A9 FS4412 实验箱、Eclipse for ARM

三、实验方法（原理、流程图）

如图所示，COM2 分别与 SP232 的 13、14 引脚相连，通过 SP3232 的 BUF_XuTXD2/UART_AUDIO_TXD 和 BUF_XuRXD2/UART_AUDIO_RXD 引脚实现 TTl 3.3V 电平转换，3.3V 电平转换再通过 U8 转变为 1.8V 电平和 CPU 通讯。这样对 EXYNOS4412 的 C8、D8 操作就可以实现在 PC 串口上显示数据。SP3232 起到变压器的作用。

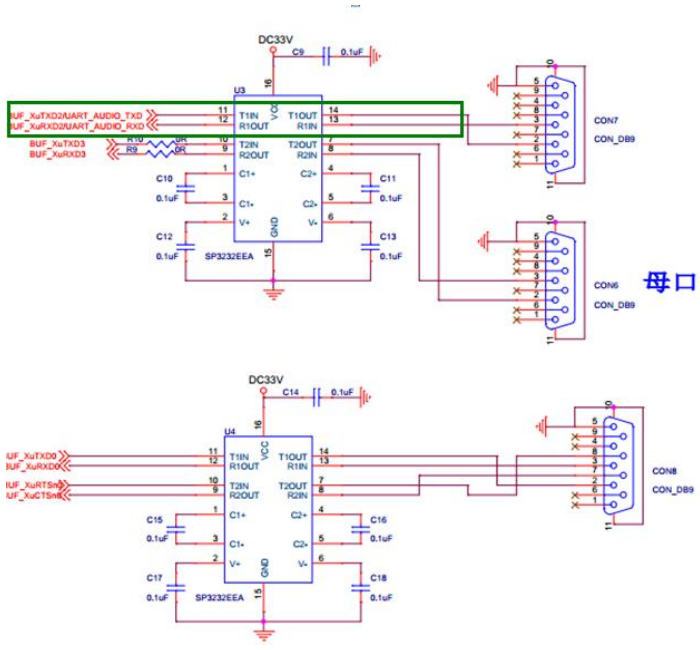


图 1 COM1 与 SP3232 连接

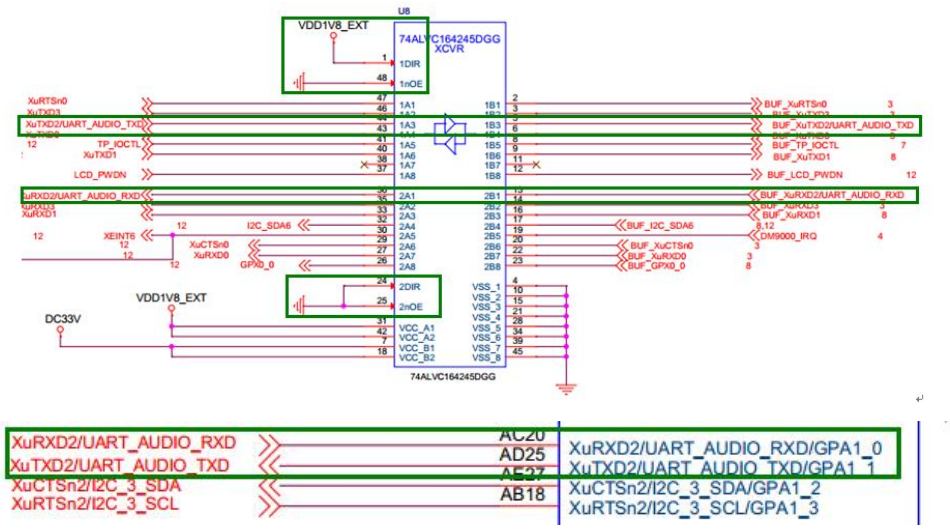


图 2 SP232 与 EXYNOS4412 相连

PC 端和 EXYNOS4412 要设置相同的串口配置，如：波特率 115200，停止位 1，数据位宽 8 位，无奇偶校验。在 EXYNOS4412 上编程实现串口配置后，向 PC 主机发送一层字符，PC 主机使用串口终端软件显示接收到的字符。

四、实验过程、步骤及内容

1、 寄存器设置

为了实现串口调试助手上显示数据，需要通过 GPA1CON 寄存器将 GPA1_0、GPA1_1 配置 UART 属性。设置 UART 串口的属性波特率、停止位、校验位等等。

2、 程序编写

```
/**********************************************************************
 * @brief       uart_init, Normal mode, No parity,One stop bit,8 data bits
 *              Buad-reate : 115200, clock srouce 100Mhz
 * @param[in]   int (ms)
 * @return      None
 **********************************************************************/
void uart_init(void)
{
```

```
    /*UART2 initialize*/
    GPA1.GPA1CON = (GPA1.GPA1CON & ~0xFF ) | (0x22); //GPA1_0:RX;GPA1_1:TX


    UART2.ULCON2 = 0x3; //Normal mode, No parity,One stop bit,8 data bits
    UART2.UCON2 = 0x5;  //Interrupt request or polling mode




    /*
     * Baud-rate 115200: src_clock:100Mhz
     * DIV_VAL = (100*10^6 / (115200*16) -1) = (54.3 - 1) = 53.3
     * UBRDIV2 = (Integer part of 53.3) = 53 = 0x35
     * UFRACVAL2 = 0.3*16 = 0x5
     * */
    UART2.UBRDIV2 = 0x35;
    UART2.UFRACVAL2 = 0x5;
}
void putc(const char data)
{
    while(!(UART2.UTRSTAT2 & 0X2));
    UART2.UTXH2 = data;
    if (data == '\n')
            putc('\r');
}
void puts(const  char  *pstr)
{
    while(*pstr != '\0')
        putc(*pstr++);
}
```

```c
unsigned char getchar()

{

    unsigned char c;

    while(!(UART2.UTRSTAT2 & 0X1));

    c = UART2.URXH2;

    return c;

}


/*-------------------------MAIN FUNCTION-----------------------------*/

/*********************************************************************

 * @brief       Main program body

 * @param[in]   None

 * @return      int

 *********************************************************************/

int main(void) {


    char c, str[] = "uart test!! \n";


    //LED

    GPX2.GPX2CON = 0x1 << 28;

    uart_init();


    while(1)

        {

            //Turn on LED

            GPX2.GPX2DAT = GPX2.GPX2DAT | 0x1 << 7;

            puts(str);

            mydelay_ms(500);

            //Turn off LED
```

```
            GPX2.GPX2DAT = GPX2.GPX2DAT & ~(0x1 << 7);

            mydelay_ms(500);

        }

    return 0;

}
```

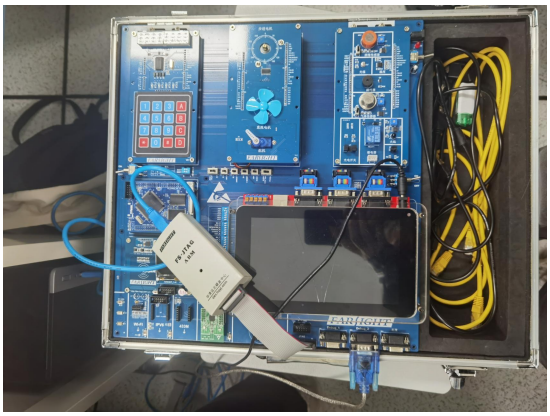
实验步骤:

1、 导入工程源码

参考 ARM 开发环境搭建部分。

光盘实验源码路径：华清远见-CORTEXA9 资料\实验代码\1. ARM 体系结构与接口技术部分\实验代码\10-UART

2、 连接好开发板及 FS-JTAG 仿真器，并且连接好配线。



3、 配置好串口终端

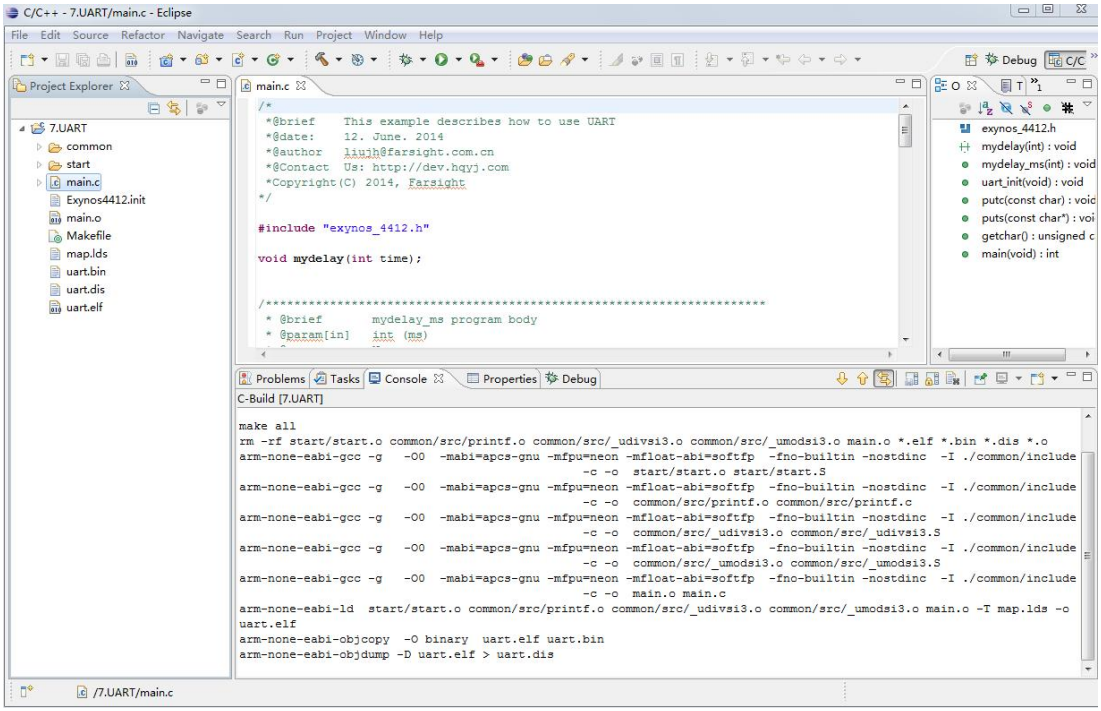
用于显示按键的值。方法参考 ARM 开发环境搭建部分。

4、仿真运行程序

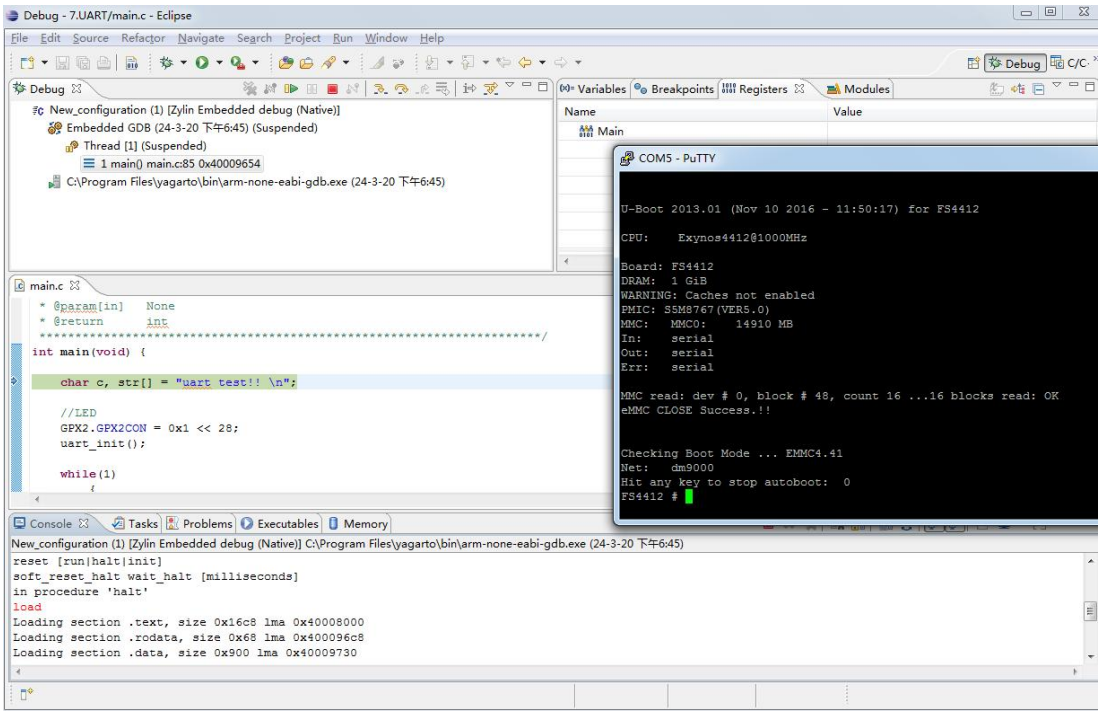
方法参考 ARM 开发环境搭建部分。

五、实验数据（现象）处理分析

1. 导入工程源码

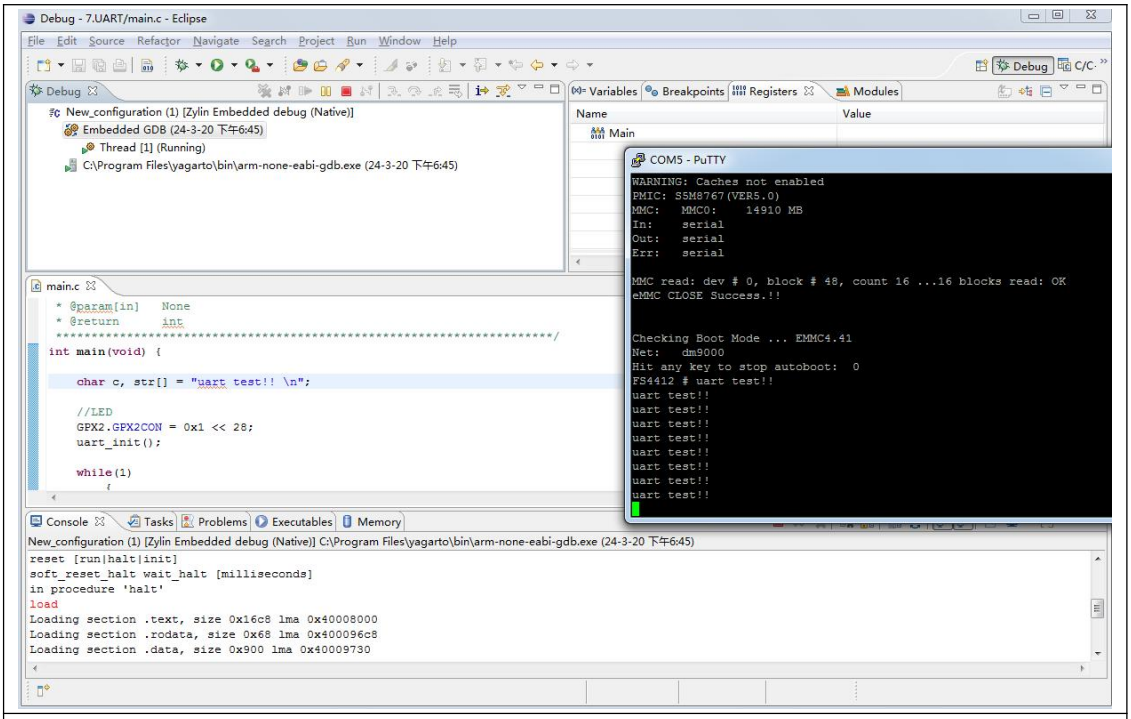


2. 配置好串口终端



3. 仿真运行程序

在串口终端上可以看到打印出的信息。



六、实验结论

在这个实验中，我们成功地利用了 EXYNOS4412 的复用引脚 XuRXD2 和 XuTXD2，通过串口与 PC 进行数据的收发，并在串口调试助手上显示了数据。通过对 COM2 与 SP232、SP232 与 EXYNOS4412 的连接，以及相应的寄存器设置和程序编写，我们确保了串口通信的正常运行。在实验过程中，我们导入了工程源码，配置了串口终端，并进行了仿真运行程序。最终，我们成功地在串口终端上观察到了打印出的信息，验证了串口通信的可靠性和正确性。这个实验不仅加深了对串口通信原理的理解，也提升了对 ARM Cortex-A9 处理器及其开发环境的熟练程度。

练习题：

1．串行通信与并行通信的概念是什么？

串行通信是指计算机与 I/O 设备之间数据传输的各位是按顺序依次一位接一位进行传送。并行通信是指计算机与 I/O 设备之间通过多条传输线交换数据，数据的各位同时进行传送。

2．同步通信与异步通信的概念及区别是什么？

概念:

异步通信,是指数据传送以字符为单位,字符与字符间的传送是完全异步的,位与位之间的传送根本上是同步的。

同步通信,是指数据传送是以数据块〔一组字符〕为单位,字符与字符之间、字符内部的位与位之间都同步。

区别:

异步通信:

〔1〕以字符为单位传送信息。

〔2〕相邻两字符间的间隔是任意长。

〔3〕因为一个字符中的波特位长度有限,所以需要的接收时钟和发送时钟只要相近就可以。

〔4〕异步方式特点就是:字符间异步,字符内部各位同步。

同步通信:

〔1〕以数据块为单位传送信息。

〔2〕在一个数据块〔信息帧〕内,字符与字符间无间隔。

〔3〕因为一次传输的数据块中包含的数据较多,所以接收时钟与发送时钟严格同步,通常要有同步时钟。

3．RS-232C 串口通信接口规范是什么？

RS-232C 标准(协议)的全称是 EIA-RS-232C 标准,定义是“数据终端设备(DTE)和数据通讯设备（DCE）之间串行二进制数据交换接口技术标准”。

4．在 EXYNOS4412 串口控制器中，哪个寄存器用来设置串口比特率？

UBRDIVn 和 UFRACVALn

5．编写一个串口程序采用中断的方式，实现向 PC 的串口终端打印一个字符串“hello”的功能。

```
#include <stdint.h>

// 定义寄存器地址
#define GPA1CON  (*((volatile uint32_t *)0x11400020))
#define ULCON2  (*((volatile uint32_t *)0x13820000))
```

```c
#define UCON2 (*((volatile uint32_t *)0x13820004))
#define UBRDIV2 (*((volatile uint32_t *)0x13820028))
#define UFRACVAL2 (*((volatile uint32_t *)0x1382002C))
#define UTRSTAT2 (*((volatile uint32_t *)0x13820010))
#define UTXH2 (*((volatile uint32_t *)0x13820020))
#define GPX2DAT (*((volatile uint32_t *)0x11000C40))

// 定义串口初始化函数
void uart_init(void) {
    // 配置 GPIO 为串口功能
    GPA1CON = (GPA1CON & ~0xFF) | 0x22;

    // 设置串口参数
    ULCON2 = 0x3;  // 正常模式，无校验，一停止位，8 数据位
    UCON2 = 0x5;   // 中断请求或轮询模式

    // 设置波特率为 115200
    UBRDIV2 = 0x35;    // 整数部分
    UFRACVAL2 = 0x5;   // 小数部分
}

// 定义发送字符函数
void putc(const char data) {
    while (!(UTRSTAT2 & 0X2));  // 等待发送缓冲区为空
    UTXH2 = data;               // 发送字符
    if (data == '\n')           // 如果发送的是换行符，则补发回车符
        putc('\r');
}

// 定义发送字符串函数
void puts(const char* pstr) {
    while (*pstr != '\0')   // 逐字符发送字符串
        putc(*pstr++);
}

// 定义中断处理函数
void UART2_Handler(void) {
    // 在此可以添加中断处理代码，但对于简单的发送操作，通常不需要额外的中断处理
}

// 主函数
int main(void) {
    // LED 初始化，若需要的话
    GPX2DAT |= 0x1 << 7;
```

```
    // 串口初始化
    uart_init();

    // 发送字符串
    puts("hello\n");

    while (1);  // 死循环，保持程序运行

    return 0;
}
```

七、教师批阅意见：

成绩评定：

教师签字：

年　　月　　日

八、备注：